A Deep Reinforcement Learning Algorithm Using Dynamic Attention Model for Vehicle Routing Problems

Bo Peng, Jiahai Wang(✉), and Zizhen Zhang

Department of Computer Science, Sun Yat-sen University, Guangzhou, China
wangjiah@mail.sysu.edu.cn

Abstract. Recent researches show that machine learning has the potential to learn better heuristics than the one designed by human for solving combinatorial optimization problems. The deep neural network is used to characterize the input instance for constructing a feasible solution incrementally. Recently, an attention model is proposed to solve routing problems. In this model, the state of an instance is represented by node features that are fixed over time. However, the fact is, the state of an instance is changed according to the decision that the model made at different construction steps, and the node features should be updated correspondingly. Therefore, this paper presents a dynamic attention model with dynamic encoder-decoder architecture, which enables the model to explore node features dynamically and exploit hidden structure information effectively at different construction steps. This paper focuses on a challenging NP-hard problem, vehicle routing problem. The experiments indicate that our model outperforms the previous methods and also shows a good generalization performance.

Keywords: Learning heuristics · Dynamic encoder-decoder architecture · Vehicle routing problem · Reinforcement learning · Neural network.

1 Introduction

Vehicle routing problem (VRP) [1] is a well-known combinatorial optimization problem in which the objective is to find a set of routes with minimal total costs. For every route, the total demand cannot exceed the capacity of the vehicle. In literature, the algorithms for solving VRP can be divided into exact and heuristic algorithms. The exact algorithms provide optimal guaranteed solutions but are infeasible to tackle large-scale instances due to high computational complexity, while the heuristic algorithms are often fast but without theoretical guarantee. Considering the trade-off between optimality and computational costs, heuristic algorithms can find a suboptimal solution within an acceptable running time for large-scale instances. However, it is non-trivial to design a good heuristic algorithm, since it requires substantial problem-specific expert knowledge and hand-crafted features. Designing a heuristic algorithm is a tedious process, can we learn a heuristic automatically without human intervention?

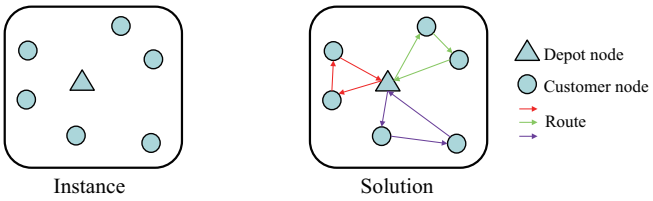

Fig. 1. A typical vehicle routing problem.

Motivated by recent advancements in machine learning, especially deep learning, there have been some works [2–7] on using end-to-end neural network to directly learn heuristics from data without any hand-engineered reasoning. Specifically, taking VRP for example, as shown in Fig. 1, the instance is a set of nodes, and the optimal solution is a permutation of these nodes, which can be seen as a sequence of decisions. Therefore, VRP can be viewed as a decision making problem that can be solved by reinforcement learning. From the perspective of reinforcement learning, typically, the state is viewed as the partial solution of instance and the features of each node, the action is the choice of next node to visit, the reward is the negative tour length, and the policy corresponds to heuristic strategy which is parameterized by a neural network. Then the policy is trained to make decisions for maximizing the reward. From the perspective of learning heuristics, given the instances from the distribution $\mathcal{S}$, a heuristics is learned to solve an unseen instance from the same distribution $\mathcal{S}$.

Recently, an attention model (AM) [7] is proposed to solve routing problems. In AM, an instance is viewed as a graph, and node features are extracted to represent such a complex graph structure, which captures the properties of a node in the context of its graph neighborhoods. Based on these node features, the solution is constructed incrementally. In AM, the node features are encoded as an embedding which is fixed over time. However, at different construction steps, the state of instance is changed according to the decision the model made, and the node features should be updated correspondingly.

This paper proposes a dynamic attention model (AM-D) with dynamic encoder-decoder architecture. The key of our improvement is to characterize each node dynamically in the context of the graph, which can explore and exploit hidden structure information effectively at different construction steps. To demonstrate the effectiveness of the proposed method, AM-D is applied to a challenging combinatorial optimization problem, vehicle routing problem. The numerical experiments indicate that AM-D performs significantly better than AM and obviously decreases the optimality gap.

This paper is structured as follows. Section 2 reviews related work. Section 3 discribes original attention model for VRP. Section 4 and 5 present our dynamic attention model for VRP and the training method, respectively. Experimental results are given in Section 6. Section 7 concludes this paper.

2 Related Work

Learning heuristic based methods proposed in last serval years can be divided into two categories in terms of types of problems solved. The first category focuses on solving permutation based combinatorial optimization problems, such as VRP and TSP. The second category solves 0-1 based combinatorial optimization problems, such as SAT and knapsack problem.

For the first category, the pointer network (PN) is introduced in [8], it takes combinatorial optimization problems as sequence to sequence problems where the input is a sequence of nodes and the output is a permutation of the input. PN overcomes the limitation that the output length depends on input by a "pointer", which is a variant of attention mechanism [9]. This sequence to sequence model [10] is trained by the supervised manner and the label is given by an approximate solver.

However, PN is sensitive to the quality of labels and optimal solutions are expensive. In [3], the neural combinatorial optimization framework is proposed to solve combinatorial optimization problems, and the REINFORCE algorithm [11] is used to train a policy modeled by PN without supervised signals. In [4], the LSTM encoder of PN is replaced by element-wise projections which are invariant to the input order and will not introduce redundant sequential information.

In [5], combinatorial optimization is taken as a graph problem, and graph embedding [12] is used to capture combinatorial structure information between nodes. The model is trained by 1-step DQN [13] which is data-efficient, and the solution is constructed by the helper function.

In [6] and [7], graph attention network [14] is used to extract the features of each node in graph structure. In [6], an explicitly forgetting mechanism is introduced to construct a solution, which only requires the last three selected nodes per step. Then the constructed solution is improved by 2OPT local search [15]. In [7], a context vector is introduced to represent the decoding context, and the model is trained by the REINFORCE algorithm with a deterministic greedy rollout baseline.

For the second category, in [16], the graph convolutional network [17, 18] is trained to estimate the likelihood, for each node in the instance, of whether this node is part of the optimal solution. In addition, the tree search is used to construct a large number of candidate solutions. In [19], GCOMB is proposed to solve combinatorial optimization problems over large graph based on graph convolutional network and Q-learning. In [20] and [21], the model is taken as a classifier. In [20], the message passing neural network [22] is trained to predict satisfiability on SAT problems. In [21], the graph neural network is used to solve decision TSP.

Since this study is targeted at solving VRP, [4, 7] are the most related work with this paper. AM proposed recently in [7] for VRP is introduced as follows.

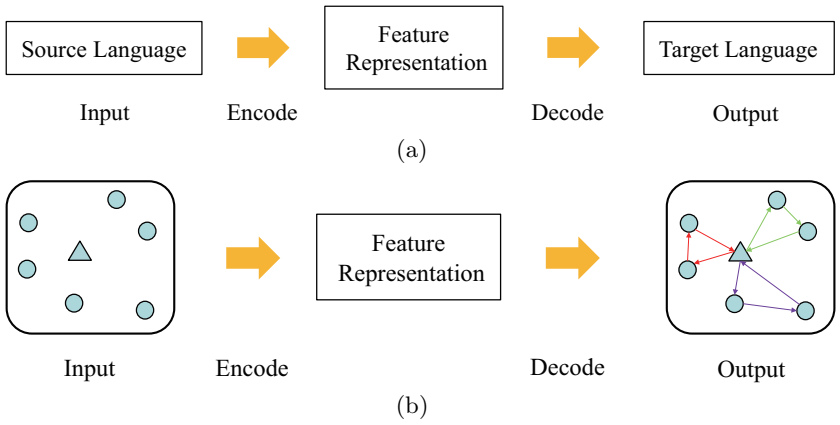

Fig. 2. (a) The encoder-decoder architecture for NMT. (b) The encoder-decoder architecture for VRP.

3 Attention Model for VRP

3.1 Problem Formulation and Preliminaries

This paper focuses on VRP. For the simplest form of the VRP, a single capacitated vehicle is responsible for delivering items to multiple customer nodes, and the vehicle must return to the depot to pick up additional items when it runs out of loads. The solution can be seen as a set of routes. In each route, it begins and ends at the depot.

Specifically, for VRP instance, the input $X = \{x_0, \ldots, x_n\}$ is a set of nodes and x_0 is the depot. Each node consists of two elements $x_i = (s_i, d_i)$, where s_i is a 2-dimensional coordinate of node i in euclidean space and d_i is its demand ($d_0 = 0$). The solution π is a sequence $\{\pi = (\pi_1, \ldots, \pi_T), \pi_t \in \{x_0, \ldots, x_n\}\}$, where each customer node is visited exactly once and the depot can be visited multiple times. T is the length of sequence π that may be varied from different solutions.

VRP can be viewed as a sequential decision making problem, and encoder-decoder architecture [10] is an effective framework for solving such kind of problems. Taking neural machine translation (NMT) for example, as shown in Fig. 2(a), the encoder extracts syntactic structure and semantic information from source language text. Then the decoder constructs target language text from the features given by encoder. Fig. 2(b) shows that the encoder-decoder architecture can also be applied to solve VRP. Firstly, the structural features of the input instance are extracted by the encoder. Then the solution is constructed incrementally by the decoder. Specifically, at each construction step, the decoder predicts a distribution over nodes, then one node is selected and appended to the end of the partial solution. Hence, corresponding to the parameters θ and input instance X, the probability of solution $p_\theta(\pi|X)$ can be decomposed by chain rule as:

$$p_\theta(\pi|X) = \prod_{t=1}^{T} p_\theta(\pi_t|X, \pi_{1:t-1}). \tag{1}$$

3.2 Encoder

In encoder, graph attention network is used to encode node features to an embedding in context of graph. It is similar to the encoder in transformer architecture [23]. Firstly, for each d_x-dimensional (for VRP, $d_x = 3$, the coordinate and demand) input node x_i, the d_h-dimensional ($d_h = 128$) initial node embedding $h_i^{(0)}$ is computed through a linear transformation with learnable parameters $W \in \mathbb{R}^{d_h \times d_x}$ and $b \in \mathbb{R}^{d_h}$, separate parameters W_0 and b_0 are used for the depot:

$$h_i^{(0)} = \begin{cases} Wx_i + b & \text{if } i \neq 0 \\ W_0 x_i + b_0 & \text{if } i = 0. \end{cases} \tag{2}$$

These initial node embeddings are fed into the first layer of graph attention network and updated $N = 3$ times with N attention layers. For each layer, it consists of two sublayers: a multi-head attention (MHA) sublayer and a fully connected feed-forward (FF) sublayer.

Multi-Head Attention Sublayer As in [23], multi-head attention is used to extract different types of information. In the layer $\ell \in \{1, \ldots, N\}$, $h_i^{(\ell)}$ is denoted as the node embedding of each node i, and the output $\{h_0^{(\ell-1)}, \ldots, h_n^{(\ell-1)}\}$ of the layer $\ell - 1$ is the input of the layer ℓ. The multi-head attention vector $\mathrm{MHA}_i^{(\ell)}(h_0^{(\ell-1)}, \ldots, h_n^{(\ell-1)})$ of each node i can be computed as:

$$q_{im}^{(\ell)} = W_m^Q h_i^{(\ell-1)}, k_{im}^{(\ell)} = W_m^K h_i^{(\ell-1)}, v_{im}^{(\ell)} = W_m^V h_i^{(\ell-1)}, \tag{3}$$

$$u_{ijm}^{(\ell)} = (q_{im}^{(\ell)})^T k_{jm}^{(\ell)}, \tag{4}$$

$$a_{ijm}^{(\ell)} = \frac{e^{u_{ijm}^{(\ell)}}}{\sum_{j'=0}^{n} e^{u_{ij'm}^{(\ell)}}}, \tag{5}$$

$$h_{im}^{'(\ell)} = \sum_{j=0}^{n} a_{ijm}^{(\ell)} v_{jm}^{(\ell)}, \tag{6}$$

$$\mathrm{MHA}_i^{(\ell)}(h_0^{(\ell-1)}, \ldots, h_n^{(\ell-1)}) = \sum_{m=1}^{M} W_m^O h_{im}^{'(\ell)}. \tag{7}$$

Here, the number of head is set $M = 8$, in each attention head $m \in \{1, \ldots M\}$, the query vector $q_{im}^{(\ell)} \in \mathbb{R}^{d_k}$, key vector $k_{im}^{(\ell)} \in \mathbb{R}^{d_k}$ and value vector $v_{im}^{(\ell)} \in \mathbb{R}^{d_v}$ are computed with parameters $W_m^Q \in \mathbb{R}^{d_k \times d_h}$, $W_m^K \in \mathbb{R}^{d_k \times d_h}$ and $W_m^V \in \mathbb{R}^{d_v \times d_h}$

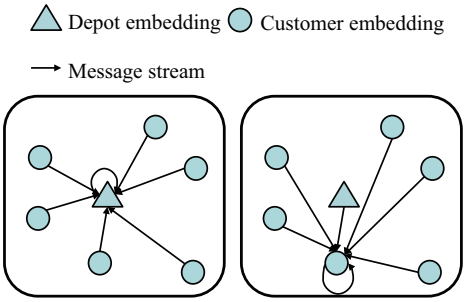

Fig. 3. The message stream in encoder. The embedding of each node is updated by aggregating the message of each node (including itself). On the left, the depot embedding is updated. On the right, the customer embedding is updated.

respectively. And the final vector is computed with $W_m^O \in \mathbb{R}^{d_h \times d_v}$ ($d_k = d_v = \frac{d_h}{M} = 16$).

Remark: the parameters W_m^Q, W_m^K and W_m^V do not share between each layer and the superscript ℓ is omitted for readability.

Feed-Forward Sublayer In this sublayer, for each node i, based on multi-head attention vector, $h_i^{(\ell)}$ is computed by skip-connection and fully connected feed-forward (FF) network. For each node i:

$$\hat{h}_i^{(\ell)} = \tanh(h_i^{(\ell-1)} + \mathrm{MHA}_i^{(\ell)}(h_0^{(\ell-1)}, \ldots, h_n^{(\ell-1)})), \tag{8}$$

$$\mathrm{FF}(\hat{h}_i^{(\ell)}) = W_1^F \mathrm{ReLu}(W_0^F \hat{h}_i^{(\ell)} + b_0^F) + b_1^F, \tag{9}$$

$$h_i^{(\ell)} = \tanh(\hat{h}_i^{(\ell)} + \mathrm{FF}(\hat{h}_i^{(\ell)})), \tag{10}$$

where $h_i^{(\ell)}$ is calculated with parameters $W_0^F \in \mathbb{R}^{d_F \times d_h}$, $W_1^F \in \mathbb{R}^{d_h \times d_F}$, $b_0^F \in \mathbb{R}^{d_F}$ and $b_1^F \in \mathbb{R}^{d_h} (d_F = 4 \times d_h)$.

After N attention layers, for each node i, the final node embedding h_i^N is calculated as:

$$h_i^N = \mathrm{ENCODE}_i^N(h_0^0, \ldots, h_n^0). \tag{11}$$

$\mathrm{ENCODE}_i^N(h_0^0, \ldots, h_n^0)$ is computed with Eqs. (3)-(10).

Fig. 3 illustrates the stream of message between nodes. By aggregating the message of each node, the embedding of each node is updated according to the attention mechanism.

3.3 Decoder

In decoder, at each construction step $t \in \{1, \ldots, T\}$, one node is selected to visit based on the partial solution $\pi_{1:t-1}$ and the embedding of each node. As in [7],

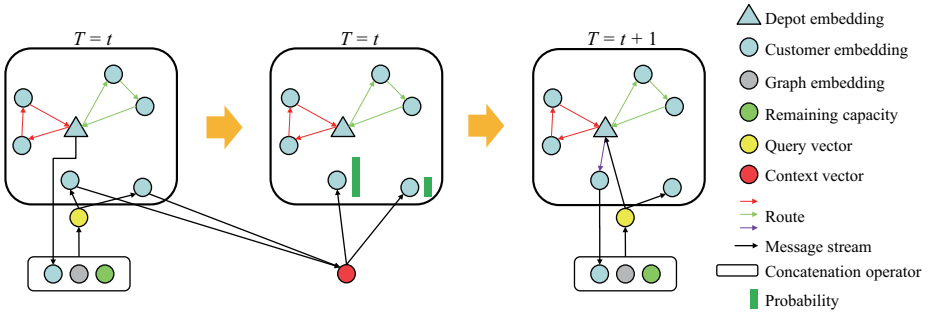

Fig. 4. The details of decoder at construction step t. At each construction step, according to the context vector and node embedding (except the nodes that violates the constraints), the decoder predicts a distribution over nodes and selects one to visit.

the context vector h_c is computed by M-head attention mechanism. Firstly, for VRP, a new vector $h_c^{'}$ is constructed as:

$$h_c^{'} = \begin{cases} [\bar{h}_t; h_0^N; D_t] & \text{if } t = 1 \\ [\bar{h}_t; h_{\pi_{t-1}}^N; D_t] & \text{if } t > 1, \end{cases} \tag{12}$$

where $[\ ;\ ]$ is concatenation operator, $h_{\pi_{t-1}}^N$ is the embedding of the node selected at construction step $t-1$, D_t is the remaining capacity of vehicle ($D_1 = D$), and $\bar{h}_t$ is the graph embedding, which is the mean vector of the embedding over nodes that have not been visited (including depot) at construction step t. Similar to the encoder, h_c is computed with a single M-head attention layer, and only a single query $q_{(c)}$ (per head) is computed (the parameters do not share with encoder):

$$q_{(c)m} = W_m^Q h_c^{'}, \quad k_{jm} = W_m^K h_j^N, \quad v_{jm} = W_m^V h_j^N, \tag{13}$$

$$u_{(c)jm} = \begin{cases} q_{(c)m}^T k_{jm} & \text{if } d_j <= D_t \text{ and } x_j \notin \pi_{1:t-1} \\ -\infty & \text{otherwise,} \end{cases} \tag{14}$$

$$a_{(c)jm} = \frac{e^{u_{(c)jm}}}{\sum_{j'=0}^{n} e^{u_{(c)j'm}}}, \tag{15}$$

$$h_{(c)m}^{'} = \sum_{j=0}^{n} a_{(c)jm} v_{jm}, \tag{16}$$

$$h_c = \sum_{m=1}^{M} W_m^O h_{(c)m}^{'}. \tag{17}$$

As shown in Eq. (14), in order to construct a feasible solution, the node that violates the constraints will be masked. For VRP, the following masking

conditions are used. First, the customer node whose demand greater than the remaining capacity of the vehicle is masked. Second, the customer node that already been visited is masked.

Remark: the depot node can be visited multiple times and it will be masked only when $\pi_{t-1} = x_0$.

Finally, the probability $p_\theta(\pi_t|X, \pi_{1:t-1})$ is computed with a single-head attention layer:

$$q = W^Q h_c, \quad k_j = W^K h_j^N, \tag{18}$$

$$u_j = \begin{cases} C \cdot \tanh(q^T k_j) & \text{if } d_j <= D_t \text{ and } x_j \notin \pi_{1:t-1} \\ -\infty & \text{otherwise,} \end{cases} \tag{19}$$

$$p_\theta(\pi_t = x_j|X, \pi_{1:t-1}) = \frac{e^{u_j}}{\sum_{j'=0}^{n} e^{u_{j'}}}, \tag{20}$$

where C is used to clip the result within $[-C, C]$ ($C = 10$). If node i is selected to visit at construction step t, the remaining capacity should be updated:

$$D_{t+1} = \begin{cases} D & \text{if } i = 0 \\ D_t - d_i & \text{otherwise.} \end{cases} \tag{21}$$

Fig. 4 illustrates the details of decoder at construction step t. According to the partial solution and node embedding, the context vector is computed by the attention mechanism. Based on the context vector and the embedding of remaining nodes, the decoder predicts a distribution over these nodes and selects one to visit.

4 Dynamic Attention Model for VRP

As mentioned in Section 3, the solution is constructed incrementally by the decoder. At different construction steps, the state of the instance is changed, and the feature embedding of each node should be updated. As shown in Fig. 5, when the model constructed a partial solution, the remaining nodes, which do not be included in the partial solution yet, can be seen as a new instance. Constructing the remaining solution is equivalent to solve this new instance. Since some nodes have already been visited, the structure of this new instance is different from the original instance. Therefore, the structure information is changed and the node features should be updated accordingly. But in vanilla encoder-decoder architecture in AM for VRP, as shown in Fig. 6(a), the feature embedding of each node is computed only once, which corresponds to the initial state of instance. This paper proposes a dynamic encoder-decoder architecture to characterize the feature embedding of each node dynamically at different construction steps.

The dynamic encoder-decoder architecture, as shown in Fig. 6(b), is similar to vanilla encoder-decoder architecture. The key difference is that the embedding of

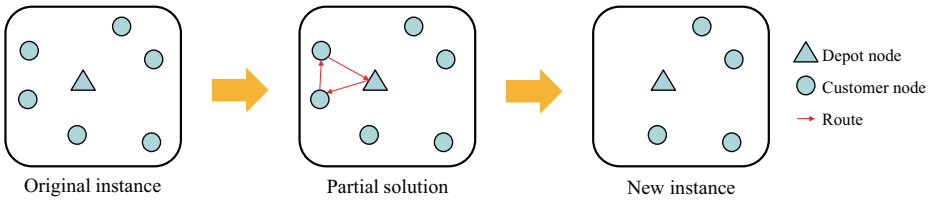

Fig. 5. The state of an instance is changed at different construction steps. When the model constructed a partial solution, the remaining nodes can be seen as a new instance. Since some nodes already been visited, the structure of this new instance is different from the original instance.

each node will be immediately recomputed when the vehicle returns to the depot. Specifically, for each node i, the embedding can be updated at construction step t as:

$$h_i^t = \begin{cases} \text{ENCODE}_i^N(h_0^0, \ldots, h_n^0) & \text{if } \pi_{t-1} = x_0 \\ h_i^{t-1} & \text{otherwise,} \end{cases} \tag{22}$$

where h_i^t is the embedding of node i at construction step t, and the layer number N is omitted. $\text{ENCODE}_i^N(h_0^0, \ldots, h_n^0)$ is similar to Eq. (11) that is computed with N M-head attention layers. The only difference is that Eq. (4) is modified. In order to reflect that the structure of instance is changed, the nodes that have been visited are masked, and Eq. (4) is modified as:

$$u_{ijm}^{(\ell)} = \begin{cases} (q_{im}^{(\ell)})^T k_{jm}^{(\ell)} & \text{if } x_j \notin \pi_{1:t-1} \text{ or } j = 0 \\ -\infty & \text{otherwise.} \end{cases} \tag{23}$$

During decoding, at each step t, the computation of Eqs. (13)-(18) is based on the latest embedding of each node $\{h_0^t, \ldots, h_n^t\}$ (the layer number N is omitted,

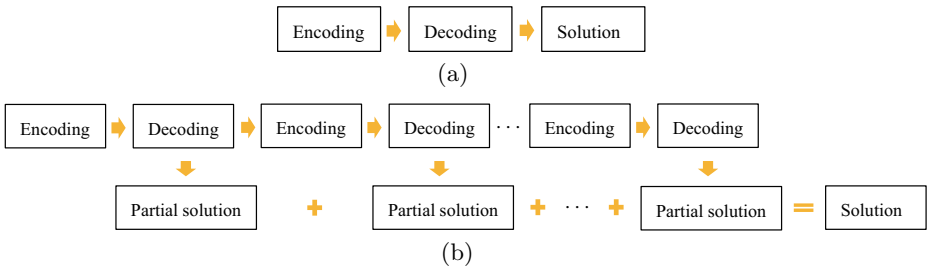

Fig. 6. The comparison between our dynamic architecture and the vanilla architecture. (a) In vanilla encoder-decoder architecture of AM for VRP, the encoder is used only once, the embedding of each node is fixed, which only can represent the initial state of the input instance. (b) In dynamic encoder-decoder architecture of AM-D, the encoder and decoder are used alternately to recode the embedding of each node and construct a partial solution.

and t is the construction step). As shown in Fig. 6(b), the entire architecture uses the encoder and decoder alternately to recode node embedding and construct a partial solution.

Given a distribution over nodes, there are two strategies to select the next node to visit. The one is sample rollout that selects a node using sampling. The other is greedy rollout that selects the node with maximum probability. The former is a stochastic policy and the latter is a deterministic policy.

5 Model Training

As in [3, 4, 6, 7], solving combinatorial optimization problem is taken as Markov Decision Processes (MDP), and AM-D is trained by policy gradient using REINFORCE algorithm [11]. Given an instance X, our training objective is the tour length of solution π. Hence, based on instance X, the gradients of parameters θ are defined as:

$$\nabla_\theta J(\theta|X) = \mathbb{E}_{\pi\sim p_\theta(.|X)}[(L(\pi|X) - b(X))\nabla_\theta \log p_\theta(\pi|X)], \tag{24}$$

where $L(\pi|X)$ is the tour length of solution π, $b(X)$ is a baseline function for estimating the expected tour length $\mathbb{E}_{\pi\sim p_\theta(.|X)}L(\pi|X)$ of instance X which can reduce the variance of gradients and accelerate convergence effectively. In this paper, as in [7], the tour length of the greedy solution, which is constructed by greedy rollout, is taken as $b(X)$.

During training, the instances are drawn from the same distribution $\mathcal{S}$. The gradients of parameters θ are approximated by Monte Carlo sampling as:

$$\nabla_\theta J(\theta) \approx \frac{1}{B}\sum_{i=1}^{B}[(L(\pi_i^s|X_i) - L(\pi_i^g|X_i))\nabla_\theta \log p_\theta(\pi_i^s|X_i)], \tag{25}$$

where B is the batch size, π_i^s and π_i^g are the solutions of instance X_i constructed by sample rollout and greedy rollout respectively. The training algorithm is described in Algorithm 1.

Algorithm 1 REINFORCE algorithm

Input: number of epochs E, steps per epoch F, batch size B
Initialize parameters θ
for epoch $= 1, \ldots, E$ **do**
  for step $= 1, \ldots, F$ **do**
    $X_i \leftarrow$ RandomInstance() for $i \in \{1, \ldots, B\}$
    $\pi_i^s \leftarrow$ SampleRollout$(p_\theta(.|X_i))$ for $i \in \{1, \ldots, B\}$
    $\pi_i^g \leftarrow$ GreedyRollout$(p_\theta(.|X_i))$ for $i \in \{1, \ldots, B\}$
    $g_\theta \leftarrow \frac{1}{B}\sum_{i=1}^{B}\left(L(\pi_i^s) - L(\pi_i^g)\right)\nabla_\theta \log p_\theta(\pi_i^s|X_i)$
    $\theta \leftarrow$ Adam(θ, g_θ)
  end for
end for

Table 1. Results on VRP. Len is the average length on test instance. Gap is the distance to state-of-the-art.

Method	VRP20, Cap30		VRP50, Cap40		VRP100, Cap50	
	Len	Gap	Len	Gap	Len	Gap
Gurobi	6.10	0.00%	-	-	-	-
LKH3	6.14	0.58%	10.38	0.00%	15.65	0.00%
RL (greedy) [4]	6.59	8.03%	11.39	9.78%	17.23	10.12%
AM (greedy) [7]	6.40	4.97%	10.98	5.86%	16.80	7.34%
AM-D (greedy)	**6.28**	**2.95%**	**10.78**	**3.85%**	**16.40**	**4.79%**
AM-D (2OPT)	6.25	2.46%	10.73	3.37%	16.27	3.96%
AM-D ($n = 20$)	-	-	11.00	5.97%	17.37	10.99%
AM-D ($n = 50$)	6.48	6.23%	-	-	16.55	5.75%
AM-D ($n = 100$)	6.65	9.02%	11.04	6.36%	-	-

6 Experiments

Experiments are conducted to investigate the performance of AM-D on VRP with node size $n = 20, 50, 100$. AM-D consists of two phases: training phase and testing phase. For each problem, in training phase, the model is trained with 30 epochs, and 10000 batches are processed in each epoch. In testing phase, the performance on 10000 test instances is reported, where the solution is constructed by greedy rollout, and the final results are the average length on all test instances.

6.1 Instances and Hyperparameters

As in [4] and [7], the instances are generated from a fixed distribution. For each node, the location are chosen randomly from the unit square $[0, 1] \times [0, 1]$, and the demand is a discrete number in $\{1, \ldots, 9\}$ chosen uniformly at random (the demand of depot is 0). The capacity of vehicle $D = 30$ for VRP with 20 customer nodes (denoted as VRP20), $D = 40$ for VRP50, $D = 50$ for VRP100, and the vehicle is located at the depot when $t = 1$. The batch size $B = 128$ and learning rate $\eta = 10^{-4}$ for both VRP20 and VRP50, $B = 108$ and $\eta = 5 \times 10^{-5}$ for VRP100. Finally, for each problem, the experiment is conducted by GPU (single 1080Ti for VRP20, VRP50, 3×1080Ti for VRP100).

6.2 Results and Discussions

Comparison Results TABLE 1 shows the results of VRP. Compared with AM, the performance of AM-D is notably improved for both VRP20 (2.02%), VRP50 (2.01%) and VRP100 (2.55%). AM-D significantly outperforms other baseline models as well.

The numerical experiments indicate that AM-D performs better than AM and other baseline methods. AM-D introduces a dynamic encoder-decoder architecture to explore structure features dynamically and exploit hidden structure information effectively at different construction steps. Hence, more hidden and useful structure information is taken into account, thereby leading to a better solution.

Generalization to Larger or Smaller Instances How does the performance of the learned heuristics generalize to test instances with larger or smaller customer node size? Experiments are conducted to investigate the generalization performance of AM-D. Specifically, the model trained with instances with 20, 50 and 100 customer nodes are denoted as AM-D ($n = 20$), AM-D ($n = 50$) and AM-D ($n = 100$), respectively. AM-D ($n = 20$) is tested on instances with 50 and 100 customer nodes, AM-D ($n = 50$) is tested on instances with 20 and 100 customer nodes, and AM-D ($n = 100$) is tested on instances with 20 and 50 customer nodes, respectively.

The results are shown at the last three rows in TABLE 1. Specifically, on the one hand, the model trained with small instances ($n = 20$) has a good performance on large instances ($n = 50$, $n = 100$), and the results even better than some baseline methods. On the other hand, the model trained with large instance ($n = 100$) performs good on small instance ($n = 20$, $n = 50$) as well. The reason why AM-D has a good generalization performance may be as follows. AM-D constructs the solution incrementally, and this process can be divided into many stages. At each stage, only a partial solution is constructed, and thus the instance is transformed to a smaller one which is easier to solve.

Combination With Local Search Local search is applied to further improve the results as in [6]. Firstly, for each instance, a solution is constructed by AM-D (greedy), then the 2OPT local search algorithm is applied to improve this solution. The resultant method is named AM-D (2OPT) and the results are shown in TABLE 1. The runtime of AM-D (greedy) and AM-D (2OPT) are also given in TABLE 2. The results indicate that the quality of the solution is improved by integrating local search, but the local search brings additional computational cost.

Table 2. Training time and testing time of AM-D. Testing time is average runtime on test instance.

	VRP20	VRP50	VRP100
Training time	14h	58h	250h
Testing time (greedy)	0.29ms	2.51ms	15.92ms
Testing time (2OPT)	0.05s	0.34s	2.21s

Discussions Machine learning and optimization are closely related, machine learning is often used as an assistant or helper component to improve the performance of solution or reduce computational costs in many optimization algorithms [24]. Totally different from these methods, AM-D is aiming to learn heuristics from data directly without human intervention. It means that knowledge or features can be extracted from the given problem instances automatically. Specifically, given an optimization problem and its instances generated from distribution $\mathcal{S}$, AM-D can learn an approximation or heuristic algorithm and solve the problem on unseen instances generated from distribution $\mathcal{S}$.

AM-D can be divided into training and testing phases like most of machine learning algorithms. The elapsed time of training and testing are shown in TABLE 2. Though the process of training is time-consuming, it is upfront, offline computation and can be seen as searching in algorithm space. Then, the trained model can be used directly to solve unseen instances without retraining from scratch, which is online even real-time computation process. Taking VRP20 for example, it takes 14 hours in training phase, but the process is one-time. Once the model has been trained, it only spends 0.29 milliseconds for solving each instance without retraining. Thus, AM-D is different from the classic heuristics, which search the solution iteratively in solution space from scratch for each instance.

The training phase of AM-D is time-consuming, thus it is trained only for problem instances with small and medium size due to the limitation of computing resources. It is promising to adopt existing parallel computing techniques to improve the computational efficiency for scaling to larger problem instances in the future.

7 Conclusion

This paper presents a dynamic attention model with dynamic encoder-decoder architecture for VRP. The key improvement is that the structure features of instances are explored dynamically, and hidden structure information is exploited effectively at different construction steps. Hence, more hidden and useful structure information is taken into account, for constructing a better solution. AM-D is tested by a challenging NP-hard problem, VRP. The results show that the performance of AM-D is better than AM and other baseline models for both problems. In addition, AM-D also shows a good generalization performance on different problem scales.

In the future, the proposed learning heuristic based method, AM-D, can be extended to solve some real-world complex VRP variants [25–30] by hybridizing with operations research method, such as VRP with time windows, which will open a new era for combinatorial optimization algorithms [2].

Acknowledgment

This work is supported by the National Key R&D Program of China (2018AAA0101203), and the National Natural Science Foundation of China (61673403, U1611262).

References

1. Madziuk, J.: New shades of the vehicle routing problem: Emerging problem formulations and computational intelligence solution methods. IEEE Transactions on Emerging Topics in Computational Intelligence **3**(3) (June 2019) 230–244
2. Bengio, Y., Lodi, A., Prouvost, A.: Machine learning for combinatorial optimization: a methodological tour dhorizon. arXiv:1811.06128 (2018)
3. Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S.: Neural combinatorial optimization with reinforcement learning. arXiv preprint arXiv:1611.09940 (2016)
4. Nazari, M., Oroojlooy, A., Snyder, L., Takác, M.: Reinforcement learning for solving the vehicle routing problem. In: Advances in Neural Information Processing Systems. (2018) 9839–9849
5. Khalil, E., Dai, H., Zhang, Y., Dilkina, B., Song, L.: Learning combinatorial optimization algorithms over graphs. In: Advances in Neural Information Processing Systems. (2017) 6348–6358
6. Deudon, M., Cournut, P., Lacoste, A., Adulyasak, Y., Rousseau, L.M.: Learning heuristics for the TSP by policy gradient. In: International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, Springer (2018) 170–181
7. Kool, W., van Hoof, H., Welling, M.: Attention, learn to solve routing problems! In: International Conference on Learning Representations. (2019)
8. Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. In: Advances in Neural Information Processing Systems. (2015) 2692–2700
9. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. In: International Conference on Learning Representations. (2015)
10. Sutskever, I., Vinyals, O., Le, Q.V.: Sequence to sequence learning with neural networks. In: Advances in Neural Information Processing Systems. (2014) 3104–3112
11. Williams, R.J.: Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning **8**(3-4) (1992) 229–256
12. Dai, H., Dai, B., Song, L.: Discriminative embeddings of latent variable models for structured data. In: International Conference on Machine Learning. (2016) 2702–2711
13. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540) (2015) 529
14. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. In: International Conference on Learning Representations. (2018)
15. Johnson, D.S.: Local optimization and the traveling salesman problem. In: International Colloquium on Automata, Languages, and Programming, Springer (1990) 446–461

16. Li, Z., Chen, Q., Koltun, V.: Combinatorial optimization with graph convolutional networks and guided tree search. In: Advances in Neural Information Processing Systems. (2018) 539–548
17. Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G.: The graph neural network model. IEEE Transactions on Neural Networks **20**(1) (2009) 61–80
18. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations. (2017)
19. Mittal, A., Dhawan, A., Medya, S., Ranu, S., Singh, A.: Learning heuristics over large graphs via deep reinforcement learning. arXiv preprint arXiv:1903.03332 (2019)
20. Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., Dill, D.L.: Learning a SAT solver from single-bit supervision. In: International Conference on Learning Representations. (2019)
21. Prates, M.O., Avelar, P.H., Lemos, H., Lamb, L., Vardi, M.: Learning to solve NP-complete problems-a graph neural network for the decision TSP. arXiv preprint arXiv:1809.02721 (2018)
22. Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E.: Neural message passing for quantum chemistry. In: Proceedings of the 34th International Conference on Machine Learning-Volume 70, JMLR. org (2017) 1263–1272
23. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems. (2017) 5998–6008
24. Song, H., Triguero, I., Özcan, E.: A review on the self and dual interactions between machine learning and optimisation. Progress in Artificial Intelligence **8**(2) (Jun 2019) 143–165
25. Yan, X., Huang, H., Hao, Z., Wang, J.: A graph-based fuzzy evolutionary algorithm for solving two-echelon vehicle routing problems. IEEE Transactions on Evolutionary Computation (2019) 1–1
26. Wang, J., Yuan, L., Zhang, Z., Gao, S., Sun, Y., Zhou, Y.: Multiobjective multiple neighborhood search algorithms for multiobjective fleet size and mix location-routing problem with time windows. IEEE Transactions on Systems, Man, and Cybernetics: Systems (2019) 1–15
27. Wang, J., Ren, W., Zhang, Z., Huang, H., Zhou, Y.: A hybrid multiobjective memetic algorithm for multiobjective periodic vehicle routing problem with time windows. IEEE Transactions on Systems, Man, and Cybernetics: Systems (2018) 1–14
28. Wang, J., Weng, T., Zhang, Q.: A two-stage multiobjective evolutionary algorithm for multiobjective multidepot vehicle routing problem with time windows. IEEE Transactions on Cybernetics **49**(7) (July 2019) 2467–2478
29. Wang, J., Zhou, Y., Wang, Y., Zhang, J., Chen, C.L.P., Zheng, Z.: Multiobjective vehicle routing problems with simultaneous delivery and pickup and time windows: Formulation, instances, and algorithms. IEEE Transactions on Cybernetics **46**(3) (March 2016) 582–594
30. Zhou, Y., Wang, J.: A local search-based multiobjective optimization algorithm for multiobjective vehicle routing problem with time windows. IEEE Systems Journal **9**(3) (Sep. 2015) 1100–1113